DLBCSPJWD01 – Project Java and Web Development

Title: Weather Dashboard Web Application

Tutor: Christian Remfert

Student: Muhammad Saad Bin Khurram

Matriculation Number:10220341

https://github.com/Saad-Khurram/DLBCSPJWD01_WeatherDashboard.git

Purpose of the Application

Real-time weather updates for any city using the OpenWeatherMap API.

Quick city search through a simple and intuitive search bar.

Save favorite cities for faster access to frequent locations.

Responsive design for seamless use on desktop, tablet, or mobile.

Target Users & Benefits

Target Users:

- Students & employees (to plan daily routines).
- Travelers (to check destination weather).
- General users who want quick updates.
- Benefits:
- Saves time by providing instant forecasts.
- Favorites list gives quick access to frequently checked locations.
- Responsive design ensures usability on desktop, tablet, and mobile.

Technical Design Choices

Frontend: HTML, CSS, JavaScript → Responsive UI.

Backend: Node.js (Express) → Handles API requests.

Database: PostgreSQL → Stores favorite cities.

API Integration: OpenWeatherMap (free API).

Dynamic Aspects:
1.Search city → fetch & display weather.
2.Save/remove cities in “Favorites” list.

Summary & Next Steps

The Weather Dashboard is a simple and user-friendly web application that helps people check real-time weather updates for any city around the world. It combines a clean, responsive front-end design with a reliable Node.js back-end connected to the OpenWeatherMap API.

With just a quick search, users can view essential weather details like temperature, humidity, and wind speed, along with easy-to-understand weather icons. The app also allows users to save their favorite cities in a personal list, making it effortless to check the weather in those places again later.

Overall, the project focuses on making weather information fast, clear, and accessible from any device whether you're at home, at work, or on the go.

In the next phase, I will begin implementing the core features and connecting the app to the weather API

Phase II

Technology	What it does in your app	Why you chose it
HTML / CSS / JavaScript	Frontend UI – search, weather card, favorites list	Required by the course; standard web technologies
Node.js	Runs the backend server	JavaScript on both frontend and backend; simple, lightweight
Express.js	Handles HTTP routes (/weather/:city, /favorites)	Minimal framework, easy to set up REST API endpoints
PostgreSQL	Stores favorite cities persistently	Relational database; reliable, free, supports conflict handling (ON CONFLICT)
Axios	Backend makes HTTP requests to OpenWeatherMap	Clean promise-based API client; better error handling than native fetch in Node
dotenv	Loads .env config (API key, DB credentials)	Keeps secrets out of source code
OpenWeatherMap API	Provides real-time weather data	Free tier available; returns JSON with temp, humidity, wind, icons
cors	Allows frontend to call backend during development	Required when frontend and backend run on different ports

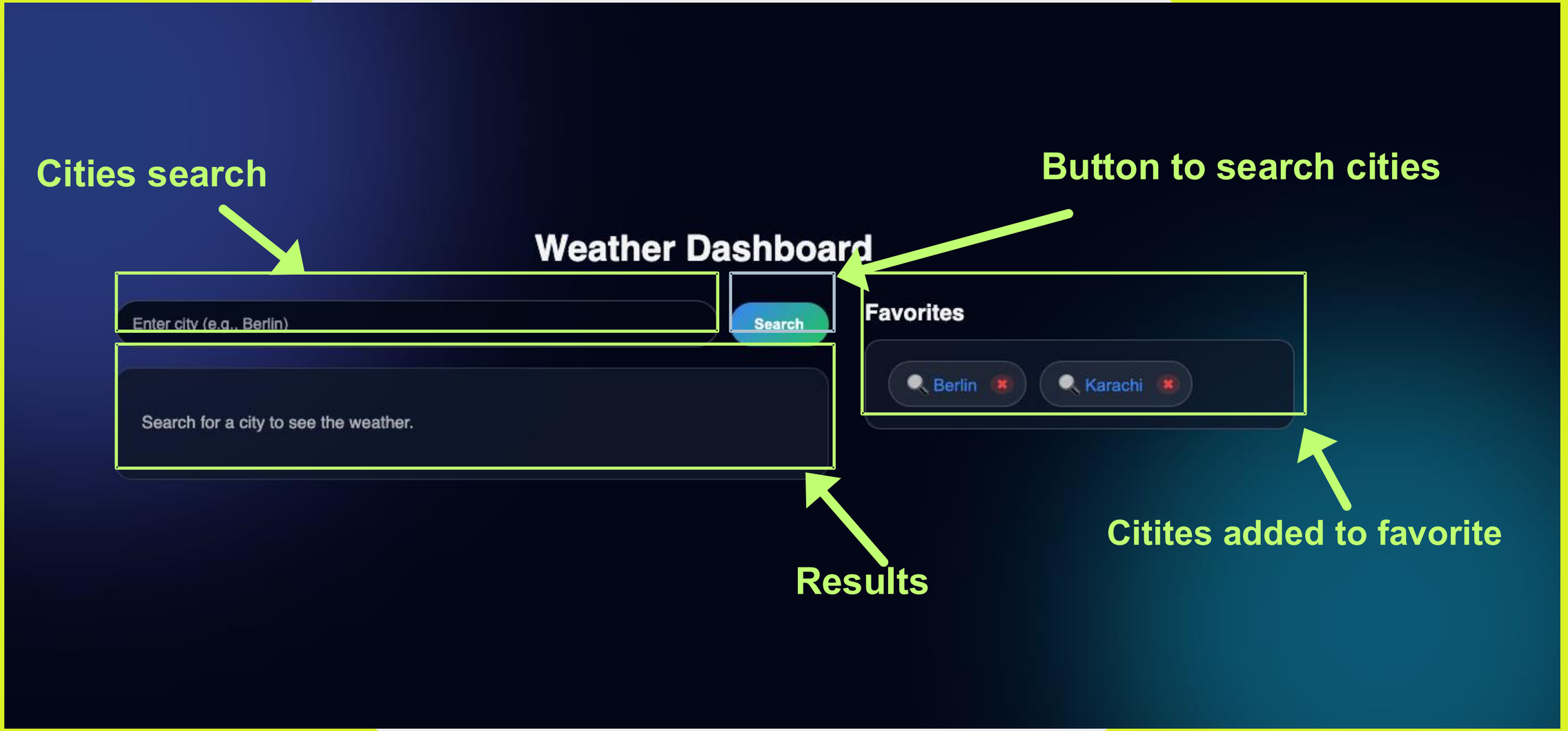
Changes from Phase 1

Area	Phase 1 Plan	What Actually Changed
Weather data displayed	Temp, humidity, wind speed, icons	Added: feels like, temp min/max, weather description, animated weather icons
UI / theming	Basic responsive layout	Dynamic card theming based on weather condition (e.g., blue for rain, yellow for clear) + loading spinner animation
City search	Simple text input → search	Added autocomplete suggestions from a cities.json list as user types
Favorites	Save/remove cities	Fully implemented with PostgreSQL – persists between sessions, sorted alphabetically
API security	OpenWeatherMap consumed by backend	Implemented correctly – API key never exposed to the frontend

#	Feature	Input	Expected Result	Actual Result	Pass/Fail
1	**City search – valid**	Type "Berlin", click Search	Weather card shows temp, humidity, wind for Berlin	Card displays Berlin weather with correct data	✓ **Pass**
2	**City search – invalid**	Type "xyzabc123", click Search	Error message: "Could not fetch weather for that city."	Error message displayed	✓ **Pass**
3	**City search – empty**	Click Search with empty input	Message: "Please enter a city."	Prompt shown, no API call made	✓ **Pass**
4	**Autocomplete**	Type "Ber" in input	Dropdown shows cities starting with "Ber" (Berlin, Bergen…)	Suggestions appear, click selects city and searches	✓ **Pass**
5	**Add to Favorites**	Search "London", click ★ Add to Favorites	"London" appears in Favorites list	London added and shown in sidebar	✓ **Pass**
6	**Duplicate favorite**	Add "London" twice	City added only once (no duplicates)	PostgreSQL ON CONFLICT DO NOTHING prevents duplicate	✓ **Pass**
7	**Remove from Favorites**	Click × next to "London"	London removed from list	Deleted from DB, list updates immediately	✓ **Pass**
8	**Click favorite city**	Click 🔍 Berlin in Favorites	Weather card updates to Berlin	Weather fetched and displayed	✓ **Pass**
9	**Responsive layout**	Resize browser to mobile width	Layout stacks vertically, input/buttons usable	Layout adjusts correctly on small screens	✓ **Pass**
10	**Persistence**	Add "Paris", restart server, reload page	Paris still in Favorites	Data persists in PostgreSQL between sessions	✓ **Pass**

#	Feature	Input	Expected Result	Actual Result	Pass/Fail
11	Dynamic card theming	Search a rainy city (e.g. "Bergen")	Weather card changes color to rain theme	Blue accent applied to card	✓ Pass
12	Loading spinner	Click Search	Spinner shows while data loads	Spinner appears, disappears when data arrives	✓ Pass
13	Responsive layout – mobile	Resize browser to 375px width	Layout stacks vertically, all elements usable	Layout adjusts correctly	✓ Pass
14	Responsive layout – tablet	Resize browser to 768px width	Layout adapts to medium screen	Elements scale and reposition correctly	✓ Pass
15	API key hidden from frontend	Open browser DevTools → Network tab	API key not visible in any frontend request	All API calls go through backend only	✓ Pass

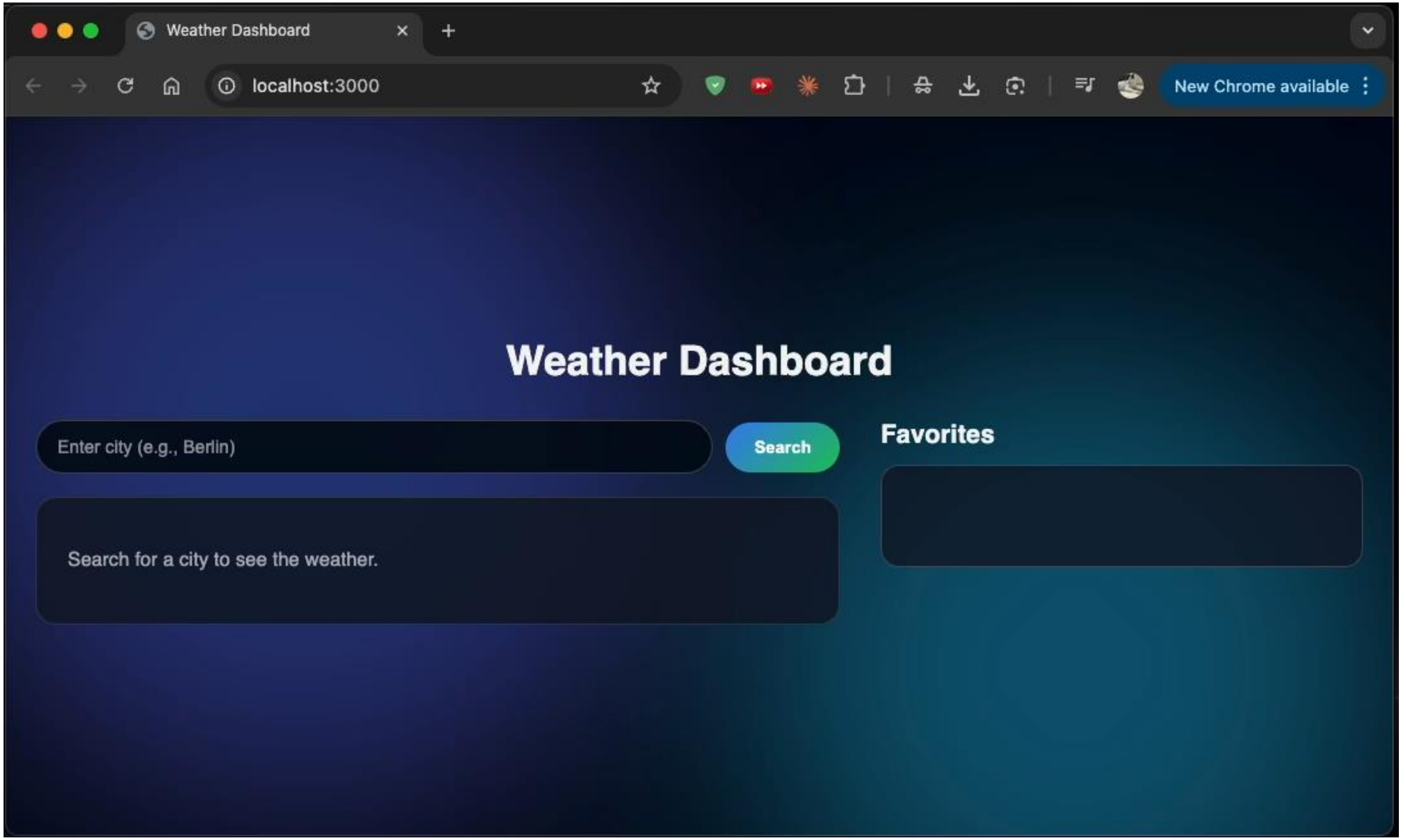
Screenshots and Screencast


Cities search
Button to search cities
Weather Dashboard
Enter city (e.g., Berlin)
Search
Favorites
Berlin
Karachi
Search for a city to see the weather.
Results
Citites added to favorite


Weather Dashboard
Berlin
Search
Favorites
Berlin
Karachi
Berlin, DE
few clouds
17°C
Feels like 17°C
HUMIDITY
55%
WIND
10.28 m/s
RANGE
↓16° ↑18°
Add to Favorites
Results

Mobile View


Weather Dashboard
London
Search
London, GB
overcast clouds
13°C
Feels like 12°C
HUMIDITY
42%
WIND
8.75 m/s
RANGE
↓12°
↑14°
Add to Favorites
Favorites
London


Weather Dashboard
localhost:3000
New Chrome available
Weather Dashboard
Enter city (e.g., Berlin)
Search
Favorites
Search for a city to see the weather.

Conclusion

The Weather Dashboard is a full-stack web application developed as part of the IU course DLBCSPJWD01. The application allows users to search for real-time weather data for any city in the world, view detailed weather information, and manage a personal list of favourite cities that persists across sessions. The frontend is built with HTML, CSS, and JavaScript, featuring a responsive layout that adapts to desktop, tablet, and mobile screens. The backend is powered by Node.js and Express, which handles all API communication with OpenWeatherMap and manages favourite cities through a PostgreSQL database.

Throughout the development phase, several enhancements were made beyond the original Phase 1 concept — including dynamic weather-based card theming, a city autocomplete search feature, animated loading states, and a more detailed weather display showing feels-like temperature, daily min/max range, and wind speed. All core features were tested and verified, covering valid and invalid inputs, data persistence, responsive behavior, and API security.

The application meets all technical requirements set by the course: at least two dynamic JavaScript interactions with the backend, responsive design, external API consumed exclusively by the backend, and full code documentation.

PHASE III

Project Overview

Abstract

The Weather Dashboard is a full-stack web application built for the IU course DLBCSPJWD01. Users can search real-time weather for any city, view detailed conditions, and manage a persistent favourites list. The frontend uses HTML, CSS, and vanilla JavaScript with a responsive layout. The backend (Node.js + Express) proxies all OpenWeatherMap requests and manages favourites via PostgreSQL.

3

Layers

Frontend · Backend · DB

15

Test Cases

All passed ✓

2+

Dynamic Interactions

JS ↔ Express API

Technology Stack

How the layers connect

Frontend

HTML5 + CSS3
Vanilla JavaScript
Responsive (mobile-first)
Glassmorphism dark theme

Backend

Node.js + Express
REST API endpoints
OpenWeatherMap proxy
node-fetch@2 (CommonJS)

Database

PostgreSQL
pgAdmin 4 GUI
Favourites table
ON CONFLICT DO NOTHING

API & Tools

OpenWeatherMap API
GitHub (version control)
VS Code editor
npm package manager

Development Journey

Step-by-step making of the Weather Dashboard

01

Project Setup

Initialised Node.js project, installed Express & pg, created folder structure, pushed to GitHub.

02

Backend API Routes

Built /weather and /favorites Express routes; secured OpenWeatherMap key server-side.

03

Database

Created PostgreSQL database & favourites table via pgAdmin; connected with pg pool.

04

Frontend UI

Designed dark glassmorphism layout with animated orbs, sidebar, and responsive CSS grid.

05

JS Interactions

Wired fetch() calls for weather search, add/remove/click favourites, and loading spinner.

06

Testing & Polish

Ran 15 test cases, fixed edge cases, added code comments, and wrote README instructions.

Challenges & Solutions

Challenge: **node-fetch ESM conflict**

→ Downgraded to node-fetch@2 to stay in CommonJS — v3 uses ES Modules which broke require().

Challenge: **PostgreSQL password forgotten**

→ Reset password via pgAdmin 4 and updated all connection strings in db.js.

Challenge: **Server crashing silently**

→ Added try/catch blocks and console.error logging to every route so errors became visible.

Challenge: **Responsive sidebar layout**

→ Used CSS media queries to collapse the sidebar into a stacked view on screens under 768 px.

System Architecture

Browser

index.html
style.css
app.js
(Vanilla JS)

fetch() → /api
Responsive UI
Favourites list

HTTP →

Node.js + Express

server.js
routes/weather.js
routes/favorites.js
db.js

/api/weather
/api/favorites
API key hidden ✓

HTTPS →

SQL →

OpenWeatherMap

External REST API
Returns JSON weather
for any city

PostgreSQL

favourites table
Persists city list
between sessions

Features Delivered

Beyond the original Phase 1 concept

City Search

Real-time weather lookup with loading spinner

Favourites (persistent)

Stored in PostgreSQL; survives server restarts

Dynamic Card Theming

Weather condition sets the card accent colour

Autocomplete Search

City suggestions appear as you type

Detailed Weather Data

Feels-like temp, min/max, wind speed

Responsive Layout

Adapts to 375 px mobile and 768 px tablet

API Key Security

Key never exposed to the browser (backend proxy)

Code Comments

Every function documented for readability

Installation & Run Instructions

Prerequisites

- Node.js (v16+)
- PostgreSQL (v14+)
- npm
- OpenWeatherMap API key
- pgAdmin 4 (recommended)

GitHub Repository

Setup Steps

1. **Clone the repo**

```
git clone <repo-url>
cd weather-dashboard
```

2. **Install dependencies**

```
npm install
```

3. **Create .env file**

```
OPENWEATHER_API_KEY=your_key
DB_PASSWORD=your_password
```

4. **Create DB table (pgAdmin)**

```
CREATE TABLE favorites
(id SERIAL, city TEXT UNIQUE);
```

5. **Start server**

```
node server.js  →  localhost:3000
```

Lessons Learned

01

Read the docs before installing

Using node-fetch v3 caused hours of debugging. Checking compatibility with CommonJS first would have saved time.

02

Error logging is essential

Silent server crashes are the hardest to debug. Adding console.error to every catch block made problems instantly visible.

03

Security by design

Keeping the API key backend-only from day one was cleaner than trying to fix it later — routing through Express made this easy.

04

Iterative development works

Building one feature at a time (search → display → favourites → responsive) kept the project manageable and testable at every step.

05

Frontend–backend separation

Treating the frontend and backend as separate systems with a clear API contract made both sides easier to develop independently.

Result & Conclusion

The Weather Dashboard successfully meets all IU course requirements: a responsive frontend, a Node.js backend, secure backend-only API consumption, at least two dynamic JS interactions, PostgreSQL persistence, and full code documentation.

The project grew beyond the original Phase 1 concept with autocomplete, dynamic theming, and animated UI — demonstrating real full-stack development from scratch.

GitHub: *https://github.com/Saad-Khurram/DLBCSPJWD01_WeatherDashboard.git*

THANKYOU